

BookMyTable Backend Explanation

Simple interview notes, ordered by execution hierarchy

Read this from top to bottom. It follows how the backend starts, receives a request, routes it, runs business logic, and stores data in MongoDB.

Backend Execution Hierarchy

- backend/package.json defines scripts: npm start and npm run dev.
- backend/server.js starts Express, middleware, database connection, routes, error handler, and listening port.
- backend/config/db.js connects to MongoDB using MONGODB_URI.
- backend/routes/index.js combines route groups under /bookings and /weather.
- backend/routes/booking.route.js maps booking URLs to controller functions.
- backend/routes/weather.route.js maps weather URL to fetchWeather.
- backend/controllers/booking.controller.js contains main business logic.
- backend/config/weather.js calls OpenWeatherMap and returns clean weather data.
- backend/models/booking.model.js defines the MongoDB Booking schema.
- backend/.env stores configuration values. backend/package-lock.json locks dependency versions.

API Endpoint Map

Method	Endpoint	Controller	Purpose
GET	/api/health	server.js	Checks whether backend is running.
POST	/api/bookings	createBooking	Creates a new reservation.
GET	/api/bookings	getAllBookings	Returns all bookings for admin dashboard.
GET	/api/bookings/available-slots?date=...	getAvailableSlots	Returns only free time slots for a date.
GET	/api/bookings/analytics	getAnalytics	Returns peak hours, popular cuisines, and total bookings.
GET	/api/bookings/:id	getBookingById	Finds booking by public bookingId.
DELETE	/api/bookings/:id	deleteBooking	Deletes booking by public bookingId.
GET	/api/weather?date=...&location=...	fetchWeather	Returns weather forecast for booking support.

1. backend/package.json

Role: Defines backend metadata, npm scripts, module style, dependencies, and dev dependencies.

Simple explanation: When you run the backend, this file tells Node which command to execute. The project uses ES modules because type is module.

Execution / logic order

- npm start -> node server.js for normal execution.
- npm run dev -> nodemon server.js for development auto-restart.
- main: server.js tells the package that server.js is the main backend file.
- dependencies are runtime packages used by the backend.

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
},
"type": "module"
```

Interview answer

I can explain that package.json is not business logic, but it is the starting configuration. It defines how the backend is launched and which libraries the server depends on.

2. backend/server.js

Role: Main backend entry file. This is the first code file to explain in execution flow.

Simple explanation: It loads environment variables, creates the Express app, enables middleware, connects to MongoDB, mounts routes under /api, adds a global error handler, and starts listening on the port.

Execution / logic order

- Imports Express, CORS, dotenv, path helpers, connectDB, and routes.
- Creates __filename and __dirname because ES modules do not provide them automatically.
- Loads backend/.env using dotenv.config().
- Creates app with express() and chooses PORT from process.env.PORT or 8082.
- Adds cors() so frontend can call backend from another port.
- Adds express.json() so JSON request bodies become req.body.
- Defines GET /api/health for quick server checking.
- Calls connectDB() to connect MongoDB.
- Mounts all imported routes at /api.
- Adds global error handler, then starts server with app.listen().

```
app.use(cors());
app.use(express.json());
app.get("/api/health", ...);
connectDB();
app.use("/api", routes);
app.listen(PORT, ...);
```

Interview answer

server.js is the bootstrap file. Every frontend API request enters through this Express server first, passes through middleware, then reaches the route and controller layers.

3. backend/config/db.js

Role: Connects the backend to MongoDB through Mongoose.

Simple explanation: It reads MONGODB_URI from environment variables. If the URI is missing or connection fails, the server logs the problem and exits instead of running with a broken database.

Execution / logic order

- Reads process.env.MONGODB_URI.
- If missing, logs an error and calls process.exit(1).
- If present, calls mongoose.connect(uri).
- Logs success after connection.
- Exports connectDB for server.js.

```
const uri = process.env.MONGODB_URI;
const conn = await mongoose.connect(uri);
```

Interview answer

Mongoose is used as the ODM. It provides schema-based modeling and methods like findOne, find, save, aggregate, and countDocuments.

4. backend/routes/index.js

Role: Main router that combines smaller route files.

Simple explanation: This file keeps server.js clean. Instead of importing every route group into server.js, it imports booking routes and weather routes here, then exports one combined router.

Execution / logic order

- Creates an Express router.
- Mounts bookingRoutes at /bookings.
- Mounts weatherRoutes at /weather.
- Exports router to server.js.

```
router.use("/bookings", bookingRoutes);  
router.use("/weather", weatherRoutes);
```

Interview answer

Because server.js mounts this router at /api, the final paths become /api/bookings and /api/weather.

5. backend/routes/booking.route.js

Role: Defines booking-related API URLs and connects them to controller functions.

Simple explanation: This route file does not contain business logic. It only maps HTTP method plus URL to the correct controller.

Execution / logic order

- POST /api/bookings calls createBooking.
- GET /api/bookings calls getAllBookings.
- GET /api/bookings/available-slots calls getAvailableSlots.
- GET /api/bookings/analytics calls getAnalytics.
- GET /api/bookings/:id calls getBookingById.
- DELETE /api/bookings/:id calls deleteBooking.

```
router.post("/", createBooking);  
router.get("/", getAllBookings);  
router.get("/available-slots", getAvailableSlots);  
router.get("/analytics", getAnalytics);  
router.get("/:id", getBookingById);  
router.delete("/:id", deleteBooking);
```

Interview answer

Routes are separated from controllers to keep the code organized. Routes decide what function runs; controllers decide what actually happens.

6. backend/routes/weather.route.js

Role: Defines the weather API route.

Simple explanation: It maps GET /api/weather to fetchWeather from booking.controller.js. This is a small route file because weather logic is handled in the controller and config/weather.js.

Execution / logic order

- Creates Express router.
- GET / maps to fetchWeather.
- Because of parent routers, final URL is /api/weather.

```
router.get("/", fetchWeather);
```

Interview answer

The weather route lets frontend request weather by query params like date and location without directly calling OpenWeatherMap from the browser.

7. backend/controllers/booking.controller.js

Role: This is the main business logic file. It creates bookings, checks slot conflicts, sends test emails, fetches availability, returns admin data, deletes bookings, fetches weather, and computes analytics.

Top-level setup

- Imports uuidv4 to create a public unique bookingId.
- Imports getWeatherForDate from config/weather.js.
- Imports nodemailer for email confirmation.
- Imports Booking model for MongoDB operations.
- Creates a module-level transporter variable for email.

initEmail()

- Creates an Ethereum test email account.
- Builds a nodemailer transporter using smtp.ethereal.email.
- Runs once when the controller file is imported.
- Important: Ethereal is for development/testing, not real production customer email.

```
let transporter;
const initEmail = async () => { ... };
initEmail();
```

sendConfirmationEmail(booking)

- Returns early if transporter is not ready.
- Builds email content using booking email, guests, date, and time.
- Sends email through nodemailer.
- Logs Ethereal preview URL so developer can view the test email.

Interview note: If asked why EMAIL_USER and EMAIL_PASS are in .env but not used here, say the current code uses Ethereal test accounts. For production, those env values could be used to configure a real SMTP provider.

createBooking(req, res)

- Reads customerEmail, bookingDate, bookingTime, numberOfGuests, and other fields from req.body.
- Normalizes bookingDate to midnight using setHours(0,0,0,0).
- Checks Booking.findOne for same date, same trimmed time, and status confirmed.
- If a booking already exists, returns 400 with This slot is already taken.
- Calls getWeatherForDate(bookingDate, Mumbai) and stores the returned weatherInfo.
- Creates a new Booking with uuid bookingId, Guest as default customerName, confirmed status, and other request data.
- Saves it with newBooking.save().
- If email looks valid, sends confirmation email.
- Returns 201 with the saved booking.

```
const exists = await Booking.findOne({
  bookingDate: targetDate,
  bookingTime: bookingTime.trim(),
  status: "confirmed",
});
```

Interview answer: This function is the core reservation flow. It validates slot uniqueness, enriches the booking with weather, saves the booking, optionally sends email, and returns the created record.

getAvailableSlots(req, res)

- Reads date from req.query.
- Defines a fixed list of allowed restaurant time slots.

- Normalizes the requested date.
- Finds confirmed bookings for that date and selects only bookingTime.
- Removes taken slots from the fixed slot list.
- Returns the remaining available slots.

getAllBookings(req, res)

- Fetches all bookings from MongoDB.
- Sorts by createdAt descending, so newest bookings appear first.
- Used by the admin dashboard list.

getBookingById(req, res)

- Reads id from URL params.
- Searches by bookingId, not MongoDB _id.
- Returns 404 if missing.
- Returns booking data if found.

deleteBooking(req, res)

- Reads id from URL params.
- Deletes the document whose bookingId matches id.
- Returns 404 if no booking is found.
- Returns success message after deletion.

fetchWeather(req, res)

- Reads date and location from query params.
- Calls getWeatherForDate(date, location).
- Returns weather JSON to frontend.

getAnalytics(req, res)

- Uses Booking.aggregate to group bookings by bookingTime and count peak hours.
- Uses Booking.aggregate to group bookings by cuisinePreference and count popular cuisines.
- Uses countDocuments to get total bookings.
- Returns peakHours, popularCuisines, totalBookings, topCuisine, and topHour.

```
Booking.aggregate([
  { $group: { _id: "$bookingTime", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
]);
```

8. backend/config/weather.js

Role: Handles weather integration with OpenWeatherMap. It keeps external API logic separate from controller logic.

- Reads OPEN_WEATHER_API_KEY from environment variables.
- If API key is missing, returns Unknown/Unavailable instead of crashing.
- Calls OpenWeather geocoding API to convert city into latitude and longitude.
- Calls OpenWeather 5-day forecast API using those coordinates.
- Finds the forecast entry matching the requested booking date.
- If exact date is not found, falls back to the first forecast item.
- Returns only temperature, condition, and description.

Interview note: OpenWeather's 5-day forecast cannot reliably answer dates far in the future. This code handles that by using fallback data, but in production you may restrict booking dates or use another forecast source.

9. backend/models/booking.model.js

Role: Defines the Booking schema and exports the Mongoose model.

Simple explanation: This file controls the shape, validation rules, defaults, and timestamps for booking documents in MongoDB.

Execution / logic order

- bookingId is required and unique. It comes from uuidv4.
- customerName is required and trimmed. Controller currently defaults it to Guest.
- customerEmail is trimmed and lowercased.
- numberOfGuests is required and must be at least 1.
- bookingDate and bookingTime are required.
- cuisinePreference and specialRequests are optional trimmed strings.
- weatherInfo stores weather JSON and defaults to empty object.
- seatingPreference is enum: indoor or outdoor, default indoor.
- status is enum: confirmed, pending, cancelled, default confirmed.
- timestamps creates createdAt and updatedAt automatically.

```
const Booking = mongoose.model("Booking", bookingSchema);  
export default Booking;
```

Interview answer

The model is important because it enforces data structure at the database layer. Controllers use this model for save, find, delete, aggregate, and count operations.

10. backend/.env

Role: Stores configuration and secrets outside source code. Do not commit real secrets in public repositories.

- MONGODB_URI: database connection string used by config/db.js.
- OPEN_WEATHER_API_KEY: API key used by config/weather.js.
- PORT: backend port used by server.js.
- EMAIL_USER and EMAIL_PASS: present in .env, but current email implementation uses Ethereum test accounts instead.

11. backend/package-lock.json and .gitignore

- package-lock.json locks exact dependency versions so installs are repeatable.
- .gitignore should prevent node_modules and secret files like .env from being committed.
- node_modules is generated by npm install and should not be explained line by line in interviews.

Complete Request Flow To Speak

When the user creates a booking from the frontend, the request goes to POST /api/bookings. server.js receives it, CORS and JSON middleware run, routes/index.js forwards /bookings to booking.route.js, and booking.route.js calls createBooking. The controller normalizes the date, checks if the same confirmed date and time already exists, fetches weather, creates a Booking document with a UUID bookingId, saves it in MongoDB, sends a test confirmation email if possible, and returns the saved booking as JSON.

Important Limitations To Be Honest About

- No authentication is currently protecting admin booking list, delete, or analytics endpoints. Production should add auth.
- Input validation is basic. Production should validate required fields, date format, email format, and allowed slots before saving.
- Slot conflict checking prevents same date/time double booking, but it does not consider table count or restaurant capacity.
- Email uses Ethereal test email, so it is good for development but not real customer delivery.
- Weather API supports only near-term forecast. Far future booking dates may use fallback weather data.
- Controllers often return res.status(500) directly, so the global error handler exists but is not heavily used with next(error).
- No automated tests are defined yet; package.json test script is still a placeholder.

Interview Questions And Short Answers

Question	Answer to give
What starts the backend?	npm run dev runs nodemon server.js, or npm start runs node server.js.
Why type: module?	It allows ES module import/export syntax instead of CommonJS require/module.exports.
Why recreate __dirname?	In ES modules, __dirname is not available by default, so fileURLToPath and path.dirname recreate it.
How is MongoDB connected?	server.js calls connectDB(), which reads MONGODB_URI and connects using mongoose.connect().
How do routes reach controllers?	server.js mounts /api, routes/index.js mounts /bookings and /weather, then route files call controller functions.
How do you prevent double booking?	createBooking checks for an existing confirmed booking with the same normalized date and trimmed time.
Why normalize date to midnight?	So the same calendar date compares consistently in MongoDB regardless of time component.
What is bookingId?	A UUID generated by uuidv4; it is public and separate from MongoDB's internal _id.
How does availability work?	It starts from a fixed list of slots, fetches confirmed bookings for the date, and removes taken slots.
How is weather used?	The backend gets forecast data from OpenWeatherMap and stores it in weatherInfo for seating or booking confirmation.
Is email real production email?	No. It uses Ethereal test SMTP, useful for development preview links. Production would use a real SMTP provider.
Any security limitation?	Admin booking and analytics routes currently have no authentication; in production, JWT/session auth should be added.

Final 60-Second Backend Explanation

The backend is a Node.js and Express API using MongoDB through Mongoose. Execution starts from package.json scripts, which run server.js. server.js loads environment variables, creates the Express app, enables CORS and JSON parsing, connects to MongoDB, mounts all API routes under /api, and starts listening. The main router splits requests into booking and weather routes. Booking routes call controller functions, where the core logic lives: creating bookings, checking slot conflicts, getting available slots, listing and deleting bookings, fetching weather, sending test emails, and generating analytics. The Booking model defines the MongoDB schema and validation rules. Configuration like MongoDB URI, weather API key, and port comes from .env.